Busca

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

18 de março de 2026

Roteiro

Introdução
- Objetivos
- Conceitos Gerais

Busca Sequencial
- Conceitos Gerais

Busca Binária
- Conceitos Gerais

Objetivos de Aprendizagem

- Compreender o problema fundamental da Busca (ou Pesquisa) na Ciência da Computação.
- Analisar e implementar os principais algoritmos de busca:
  - Busca Sequencial (Linear).
  - Busca Binária.
- Avaliar a complexidade assintótica (tempo e espaço) de cada estratégia.
- Discutir aplicações reais e adequação de estruturas de dados para otimização de buscas.

Objetivos de Aprendizagem

- Compreender o problema fundamental da Busca (ou Pesquisa) na Ciência da Computação.
- Analisar e implementar os principais algoritmos de busca:
  - Busca Sequencial (Linear).
  - Busca Binária.
- Avaliar a complexidade assintótica (tempo e espaço) de cada estratégia.
- Discutir aplicações reais e adequação de estruturas de dados para otimização de buscas.

Objetivos de Aprendizagem

- Compreender o problema fundamental da Busca (ou Pesquisa) na Ciência da Computação.
- Analisar e implementar os principais algoritmos de busca:
  - Busca Sequencial (Linear).
  - Busca Binária.
- Avaliar a complexidade assintótica (tempo e espaço) de cada estratégia.
- Discutir aplicações reais e adequação de estruturas de dados para otimização de buscas.

Objetivos de Aprendizagem

- Compreender o problema fundamental da Busca (ou Pesquisa) na Ciência da Computação.
- Analisar e implementar os principais algoritmos de busca:
  - Busca Sequencial (Linear).
  - Busca Binária.
- Avaliar a complexidade assintótica (tempo e espaço) de cada estratégia.
- Discutir aplicações reais e adequação de estruturas de dados para otimização de buscas.

A Importância do Estudo de Buscas

O Problema da Busca

Dado um conjunto de elementos, onde cada um é identificado por uma **chave** (identificador único), o objetivo é localizar eficientemente o registro correspondente a uma chave específica de pesquisa.

- É uma das tarefas mais intrínsecas e recorrentes no processamento de dados.
- O desempenho da busca é afetado pela estrutura de dados subjacente:
  - Estruturas Lineares: Vetores, Listas, Pilhas.
  - Estruturas Não-Lineares: Grafos, Árvores.
- A **ordenação prévia** atua como redutor da complexidade temporal da busca.

A Importância do Estudo de Buscas

O Problema da Busca

Dado um conjunto de elementos, onde cada um é identificado por uma **chave** (identificador único), o objetivo é localizar eficientemente o registro correspondente a uma chave específica de pesquisa.

- É uma das tarefas mais intrínsecas e recorrentes no processamento de dados.
- O desempenho da busca é afetado pela estrutura de dados subjacente:
  - Estruturas Lineares: Vetores, Listas, Pilhas.
  - Estruturas Não-Lineares: Grafos, Árvores.
- A **ordenação prévia** atua como redutor da complexidade temporal da busca.

Nomenclatura Técnica

Tabela ou Arquivo

Qualquer estrutura de dados empregada para o armazenamento de informações. É composta por um conjunto de elementos fundamentais denominados **registros**.

- **Chave de Busca (k):** Atributo principal (ou conjunto) utilizado para referenciar e distinguir os registros nas consultas.
- A Tabela e o mecanismo podem ser implementados via:
  - Vetores e matrizes (arranjo fixo e indexado).
  - Listas Dinâmicas (encadeamento simples ou duplo).
  - Variáveis Estruturadas Arbóreas (Nós).

Nomenclatura Técnica

Tabela ou Arquivo

Qualquer estrutura de dados empregada para o armazenamento de informações. É composta por um conjunto de elementos fundamentais denominados **registros**.

- **Chave de Busca (k):** Atributo principal (ou conjunto) utilizado para referenciar e distinguir os registros nas consultas.
- A Tabela e o mecanismo podem ser implementados via:
  - Vetores e matrizes (arranjo fixo e indexado).
  - Listas Dinâmicas (encadeamento simples ou duplo).
  - Variáveis Estruturadas Arbóreas (Nós).

Natureza Foco: Busca Interna x Externa

- **Busca Interna:** O volume de dados manipulados é moderado, fazendo com que o conjunto total de elementos ajuste-se completamente à memória principal (RAM).
- **Busca Externa:** Volume extensivo, necessitando consultas a bases presentes em memória secundária (ex: Árvore **B** e Banco de Dados **SQL**).

Nosso Foco

Nas próximas seções, abordam-se técnicas de **buscas internas** sob coleções lineares, priorizando o equilíbrio do custo computacional.

Natureza Foco: Busca Interna x Externa

- **Busca Interna:** O volume de dados manipulados é moderado, fazendo com que o conjunto total de elementos ajuste-se completamente à memória principal (RAM).
- **Busca Externa:** Volume extensivo, necessitando consultas a bases presentes em memória secundária (ex: Árvore **B** e Banco de Dados **SQL**).

Nosso Foco

Nas próximas seções, abordam-se técnicas de **buscas internas** sob coleções lineares, priorizando o equilíbrio do custo computacional.

Busca Sequencial - Fundamentos

- Também conhecida na literatura como *Busca Linear*.
- Configura-se como a estratégia metodológica natural e elementar perante problemas de busca sem ordenação assumida.
- **Mecânica:** Partindo de registros armazenados em posições contíguas (ou iteráveis linearmente), o algoritmo varre o domínio exaustivamente até localizar a chave procurada, ou atingir o fundo da coleção.

Comportamento - Vetor Não Ordenado

Exemplo: Busca pela chave **37**.

12	25	8	37	99	14
0	1	2	3	4	5

Comportamento - Vetor Não Ordenado

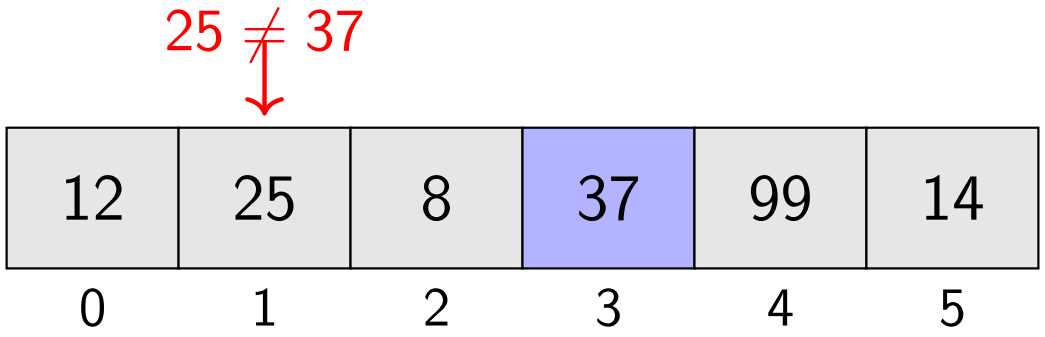
Exemplo: Busca pela chave **37**.

$12 \neq 37$

12	25	8	37	99	14
0	1	2	3	4	5

Comportamento - Vetor Não Ordenado

Exemplo: Busca pela chave **37**.



Comportamento - Vetor Não Ordenado

Exemplo: Busca pela chave **37**.

$8 \neq 37$

12	25	8	37	99	14
0	1	2	3	4	5

Comportamento - Vetor Não Ordenado

Exemplo: Busca pela chave **37**.

Encontrado ($i = 3$)

12	25	8	37	99	14
0	1	2	3	4	5

Algoritmo: Vetor Não Ordenado

Busca Sequencial Universal

```
function BuscaSequencial(A, n, x)
    for i ← 0 ate n − 1 do
        if A[i] == x then
            retorne i                    ▷ Elemento foi encontrado
        end if
    end for
    retorne −1                           ▷ Condição de falha
end function
```

Comportamento - Vetor Ordenado

- Se a estrutura tiver sido previamente **ordenada**, a busca linear pode prover rejeição precoce em certas iterações.
- Em uma coleção ordenada *crescentemente*, as verificações sucessivas podem ser finalizadas impreterivelmente tão logo processarmos valores excedentes à chave investigada.
- **Impacto:** Reduz pela metade as asserções de buscas por chaves que não existam na origem.

Comportamento - Vetor Ordenado

- Se a estrutura tiver sido previamente **ordenada**, a busca linear pode prover rejeição precoce em certas iterações.
- Em uma coleção ordenada *crescentemente*, as verificações sucessivas podem ser finalizadas impreterivelmente tão logo processarmos valores excedentes à chave investigada.
- **Impacto:** Reduz pela metade as asserções de buscas por chaves que não existam na origem.

Algoritmo: Vetor Ordenado

Busca Sequencial Otimizada Antecipada

```
function BuscaSequencialOrd(A, n, x)
    for i ← 0 ate n − 1 do
        if A[i] == x then
            retorne i
        else if A[i] > x then
            retorne −1        ▷ Parada por superioridade limitante
        end if
    end for
    retorne −1
end function
```

Análise Assintótica Espaço-Temporal

Complexidade Teórica da Busca Sequencial

Dado um vetor com dimensão computacional n:

- **Melhor Caso ($O(1)$):** A chave procurada é o primeiro índice da arrumação.
- **Pior Caso ($O(n)$):** Varredura plena inibe os recursos; a chave alocam-se na extremidade ou é displicente e omissa em relação à classe.

Busca Binária - Divisão e Conquista

- Requisito obrigatório: **A coleção deve estar categoricamente ordenada** e permitir acesso em tempo constante a quaisquer dos seus índices (Vetores $O(1)$).
- **Filosofia Lógica:**
  1. Compara-se a chave com o componente central do sub-arranjo.
  2. Se for uma concordância perfeita, interrompe-se com a resposta.
  3. Se o elemento é menor que o elemento central, descarta-se a metade superior.
  4. Se maior, descarta-se a metade inferior.

Busca Binária - Divisão e Conquista

- Requisito obrigatório: **A coleção deve estar categoricamente ordenada** e permitir acesso em tempo constante a quaisquer dos seus índices (Vetores $O(1)$).
- **Filosofia Lógica:**
  1. Compara-se a chave com o componente central do sub-arranjo.
  2. Se for uma concordância perfeita, interrompe-se com a resposta.
  3. Se o elemento é menor que o elemento central, descarta-se a metade superior.
  4. Se maior, descarta-se a metade inferior.

Comportamento Progressivo

Exemplo: Busca pela chave **44**.

Encontrado $(A[meio] == 44)$

↓

esq meio dir

5	12	23	29	44	58	70	81	95
0	1	2	3	4	5	6	7	8

Comportamento Progressivo

Exemplo: Busca pela chave **44**.

Encontrado $(A[meio] == 44)$

↓

esq meio dir

5	12	23	29	44	58	70	81	95
0	1	2	3	4	5	6	7	8

Comportamento Transicional e Iterativo

Exemplo 2: Busca pela chave **23**.

meio: $44 > 23 \rightarrow$ Ignorar direita

5	12	23	29	44	58	70	81	95

Comportamento Transicional e Iterativo

Exemplo 2: Busca pela chave **23**.

meio: $12 < 23 \rightarrow$ Ignorar esquerda

5	12	23	29	44	58	70	81	95

Comportamento Transicional e Iterativo

Exemplo 2: Busca pela chave **23**.

$A[meio] == 23$. **Encontrado!**

5	12	23	29	44	58	70	81	95

Código Binário Estruturado

Busca Binária Iterativa Clássica

```
function BuscaBinaria(A, n, x)
    esq ← 0
    dir ← n − 1
    while esq ≤ dir do
        meio ← ⌊(esq + dir)/2⌋
        if A[meio] == x then
            retorne meio
        else if A[meio] < x then
            esq ← meio + 1
        else
            dir ← meio − 1
        end if
    end while
```

Análise Assintótica Espaço-Temporal

Complexidade Logarítmica

Graças à divisão em sub-problemas paralelos isolados, o número rigoroso de quebras recursivas e comparações maximiza-se estritamente à progressão $\log_2 n$.

Pior Caso ($O(\log n)$): Reduz substancialmente a busca computável, conferindo escala planetária.

Melhor ($O(1)$) e Médio ($O(\log n)$): Comportamentos matemáticos invariantes.

- **Desvantagem:** O engessamento ordenatório e as restrições inatas das manipulações dinâmicas estruturais indexadas.

Análise Assintótica Espaço-Temporal

Complexidade Logarítmica

Graças à divisão em sub-problemas paralelos isolados, o número rigoroso de quebras recursivas e comparações maximiza-se estritamente à progressão $\log_2 n$.

Pior Caso ($O(\log n)$): Reduz substancialmente a busca computável, conferindo escala planetária.

Melhor ($O(1)$) e Médio ($O(\log n)$): Comportamentos matemáticos invariantes.

- **Desvantagem:** O engessamento ordenatório e as restrições inatas das manipulações dinâmicas estruturais indexadas.

Busca

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br

[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

18 de março de 2026